

1. User-Level Threads (ULT) vs Kernel-Level Threads (KLT)

First, why do we need this distinction?

When you create threads, **who manages them?**

There are two possibilities:

- The **thread library** (user space)
- The **Operating System kernel**

That's where User-Level Threads and Kernel-Level Threads come from.

User-Level Threads (ULT)

In ULT, **the kernel doesn't know that multiple threads exist**.

The kernel only sees **one process**.



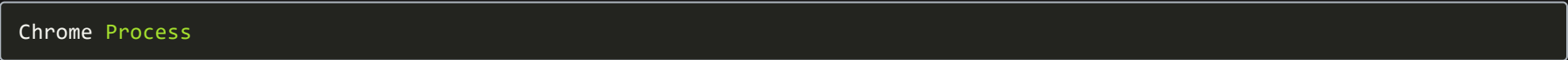
The thread library (e.g., Green Threads, older Java implementations) schedules the threads itself.

Example

Suppose Chrome has:

- UI Thread
- Network Thread
- Render Thread

The OS only knows:



Internally, Chrome switches between its threads.

Advantages

1. Fast Thread Creation

No system call needed.

Everything happens in user space.

2. Fast Context Switching

The kernel isn't involved.

Switching is simply changing:

- Stack
- Registers
- PC

3. Portable

Works without OS support.

Disadvantages

Blocking Problem

Suppose:

Thread A does

```
read(file);
```

The kernel thinks:

```
"Chrome process is waiting."
```

So the **entire process blocks**, including Threads B and C.

No True Parallelism

Even on an 8-core CPU,

the kernel only sees

```
Chrome Process
```

Only one CPU core can execute it at a time.

Kernel-Level Threads (KLT)

Now the OS knows every thread.

```
Kernel

Thread A

Thread B

Thread C
```

Each thread has its own kernel scheduling information.

Advantages

Blocking Doesn't Affect Others

Suppose Thread A waits for disk.

Thread B can still execute.

True Parallelism

On four cores:

```
Core 1 → Thread A

Core 2 → Thread B

Core 3 → Thread C
```

Threads can actually run simultaneously.

Better Scheduling

The OS schedules every thread independently.

Disadvantages

Slower Creation

Every thread creation needs a kernel call.

Slower Context Switching

Kernel participation makes switching more expensive than pure user-space thread switches.

Comparison

Feature	User-Level Threads	Kernel-Level Threads	Managed By User library	OS Kernel	Kernel aware?	☒ No ☐ Yes	Thread
Creation	Fast	Slower	Context Switch	Fast	Slower	Blocking I/O	Blocks whole process
Blocks	whole process	only that thread	Parallelism	No true parallelism	True parallelism		

2. Thread Pool

Problem

Suppose a web server receives

```
1000 Requests
```

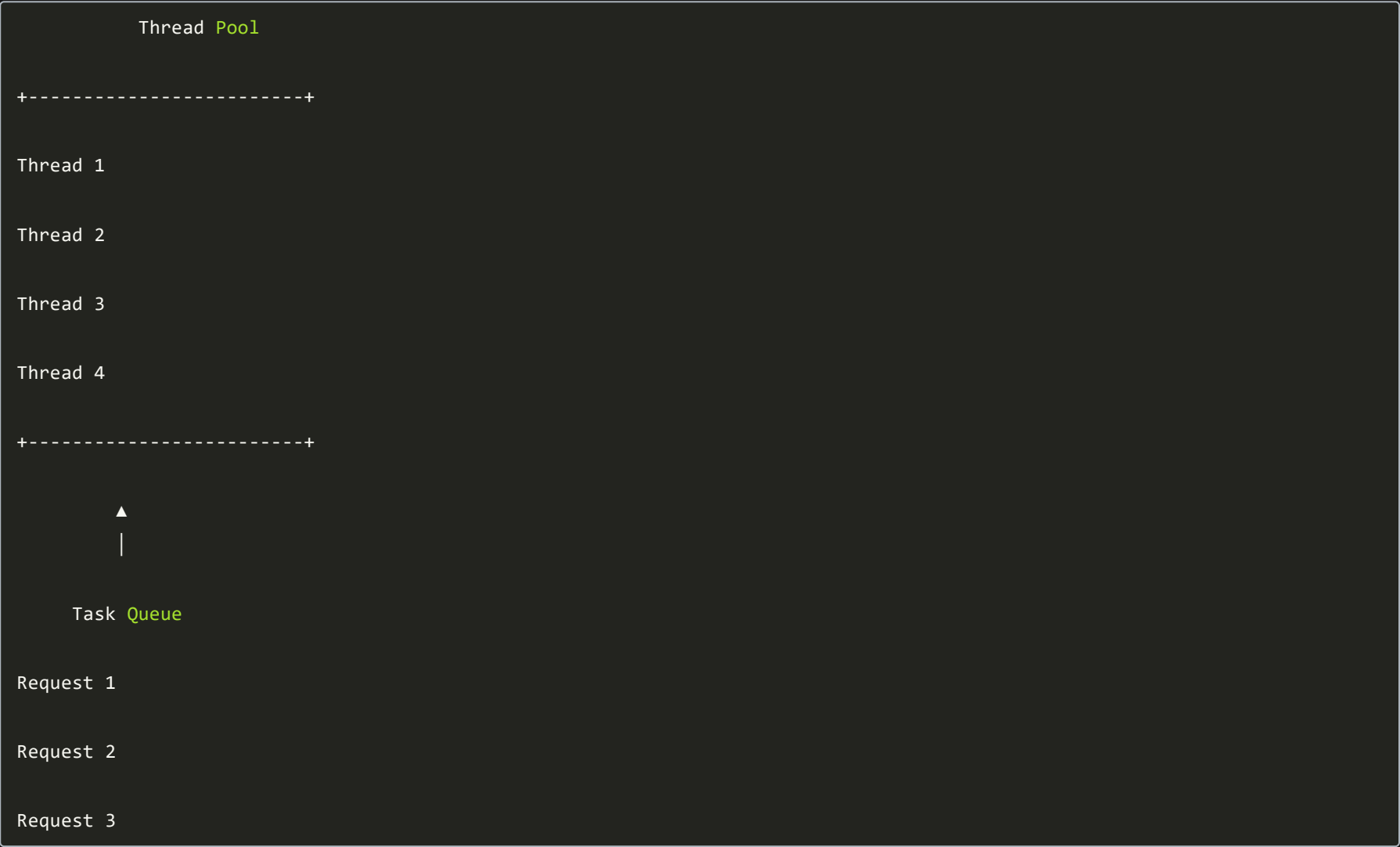
Without a thread pool:



Creating thousands of threads repeatedly is expensive.

Solution

Create a fixed number of worker threads once.



Workers continuously take tasks from the queue.

No new thread creation every time.

Advantages

Faster

Threads already exist.

No creation overhead.

Lower Memory Usage

Instead of

```
1000 Threads
```

have

```
20 Threads
```

Better CPU Utilization

The OS isn't constantly creating and destroying threads.

Real Examples

Web servers

```
Nginx

Apache

Tomcat
```

All use worker/thread pools.

Java:

```
ExecutorService
```

C++:

```
std::thread pool libraries
```

3. Concurrency vs Parallelism

This is one of the most misunderstood interview questions.

Concurrency

Multiple tasks **make progress during the same time period**.

They don't have to run at exactly the same instant.

Example:

Single CPU



Very fast switching.

Looks simultaneous.

Actually one at a time.

Restaurant Analogy

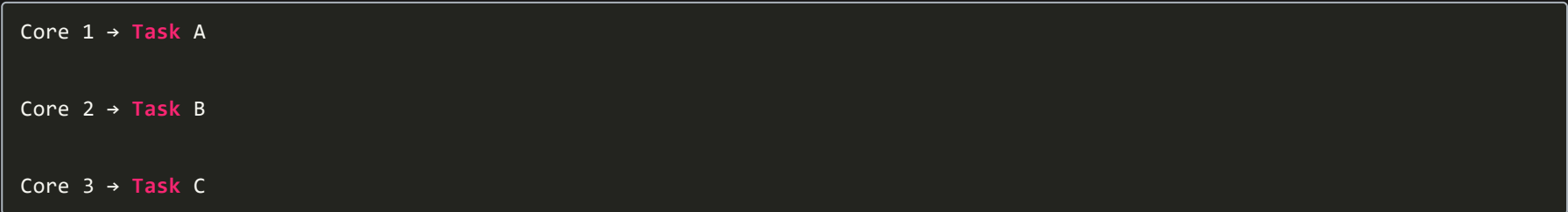
One chef.



One chef.
Many tasks.
That's concurrency.

Parallelism

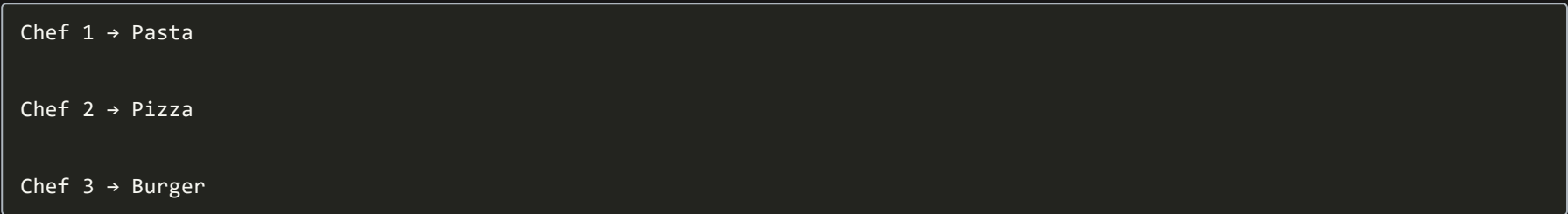
Tasks literally execute simultaneously.
Need multiple CPU cores.



Everyone works at the same instant.

Restaurant

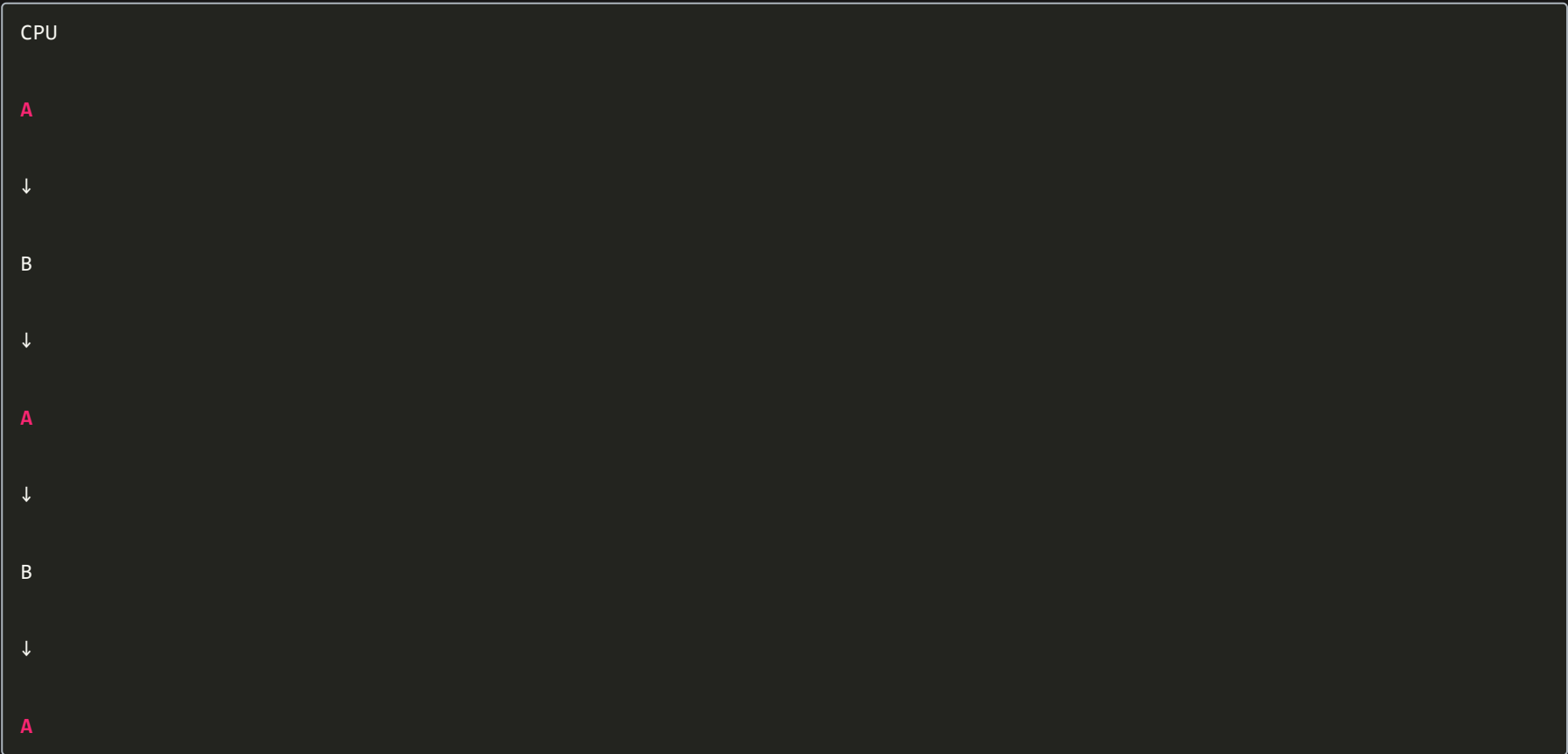
Three chefs.



Everything cooks simultaneously.

Visual

Concurrency



Parallelism

Core 1 → A
Core 2 → B
Core 3 → C

Key Difference

Concurrency	Parallelism	Multiple tasks make progress	Multiple tasks execute at the same instant	Can happen on one core	Requires multiple cores (or processors)	Achieved by context switching	Achieved by multiple execution units	Focuses on responsiveness	Focuses on performance and throughput
-------------	-------------	------------------------------	--	------------------------	---	-------------------------------	--------------------------------------	---------------------------	---------------------------------------

Interview Trick Question

Can you have concurrency without parallelism?

☒ Yes.
Single-core CPU.
Rapid context switching.

Can you have parallelism without concurrency?

In most practical software systems, **parallel execution is considered a form of concurrency** because multiple tasks are active at the same time. So the common interview answer is **No**—parallelism is a subset of concurrency.

Interview Answers

User-Level vs Kernel-Level Threads

User-level threads are managed entirely by a user-space thread library, so they are fast to create and switch but cannot achieve true parallelism and a blocking system call can block the entire process. Kernel-level threads are managed and scheduled by the operating system, allowing true parallel execution on multiple CPU cores and preventing one blocked thread from stopping others, at the cost of higher creation and context-switch overhead.

Thread Pool

A thread pool is a collection of pre-created worker threads that repeatedly take tasks from a shared queue. Reusing threads avoids the overhead of creating and destroying a thread for every request, improves CPU utilization, limits resource consumption, and is widely used in web servers and asynchronous task execution frameworks.

Concurrency vs Parallelism

Concurrency is about **dealing with multiple tasks** by making progress on each, often through context switching on a single core. Parallelism is about **doing multiple tasks at exactly the same time** using multiple CPU cores. Concurrency improves responsiveness and resource utilization, while parallelism improves execution speed by performing work simultaneously.